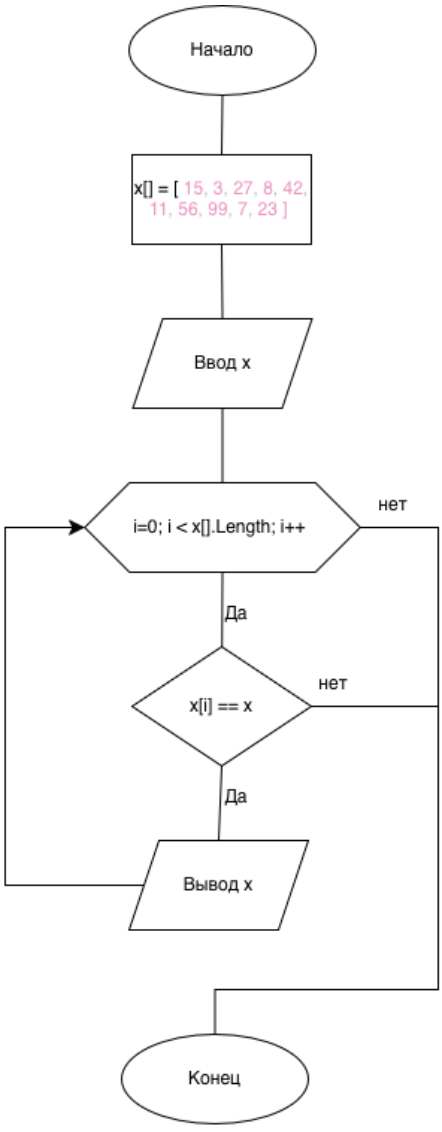
1. Постановка задачи.

«Поиск в массиве»: Задан массив из N чисел. Пользователь вводит искомое число X. Алгоритм проверяет, есть ли X в массиве, и выводит индекс первого найденного элемента или сообщение «Не найдено».

2. Контрольный пример.

| **Входные данные** | **Значения** |
|---|---|
| X | 15, 50, abc |
| **Выходные данные** | **Значения** |
| Сообщение | Индекс 0, не найдено, некорректный ввод |

3. Блок схема.



4. Текст программы.

```
int[] array = { 15, 3, 27, 8, 42, 11, 56, 99, 7, 23 };

Console.Write("массив: ");
foreach (int num in array)
    Console.Write(num + " ");

Console.WriteLine("\n");

Console.Write("X: ");
if (!int.TryParse(s: Console.ReadLine(), out int x))
{
    Console.WriteLine("некорректный ввод.");
    Program.Pause();
    return;
}

int index = -1;
for (int i = 0; i < array.Length; i++)
{
    if (array[i] == x)
    {
        index = i;
        break;
    }
}
```

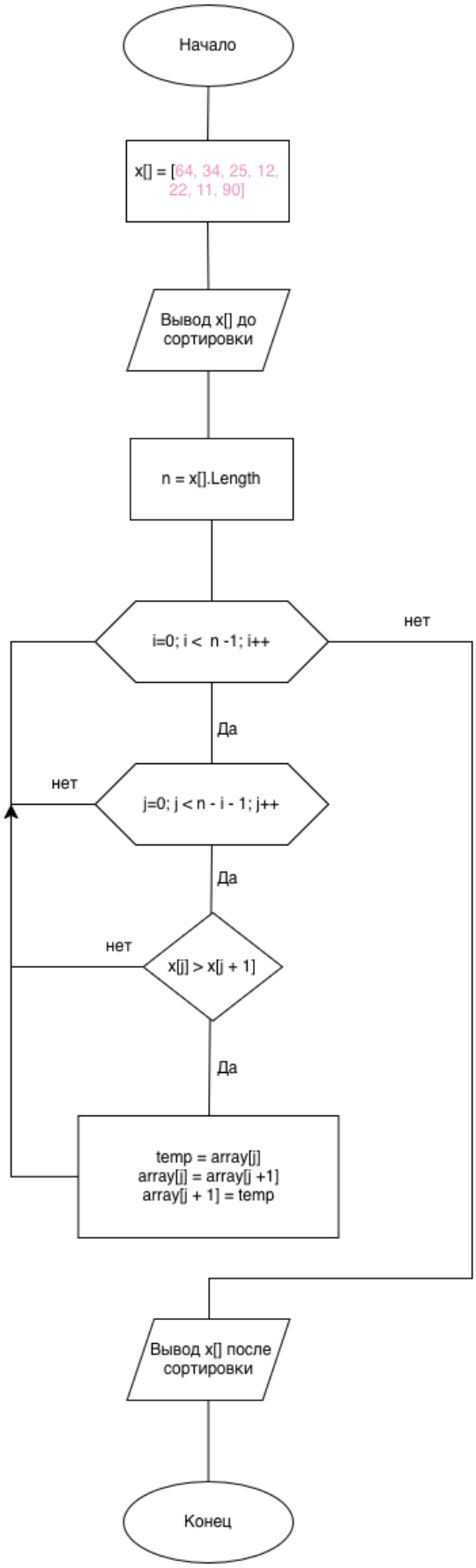
1. Постановка задачи.

«Сортировка»: Изобразите блок-схему алгоритма сортировки массива методом пузырька (Bubble Sort) или выбора (Selection Sort). и Написать программу

2. Контрольный пример.

| **Входные данные** | **Значения** |
|---|---|
| Array | {3, 1, 2} |
| **Выходные данные** | **Значения** |
| Array | {1, 2, 3} |

3. Блок схема.


Начало
x[] = [64, 34, 25, 12, 22, 11, 90]
Вывод x[] до сортировки
n = x[].Length
i=0; i < n -1; i++
нет
Да
j=0; j < n - i - 1; j++
нет
Да
x[j] > x[j + 1]
нет
Да
temp = array[j]
array[j] = array[j +1]
array[j + 1] = temp
Вывод x[] после сортировки
Конец

4. Текст программы.

```
int[] array = { 64, 34, 25, 12, 22, 11, 90 };

Console.Write("до:  ");
PrintArray(array);

int n = array.Length;

for (int i = 0; i < n - 1; i++)
{
    for (int j = 0; j < n - i - 1; j++)
    {
        if (array[j] > array[j + 1])
        {
            int temp = array[j];
            array[j] = array[j + 1];
            array[j + 1] = temp;
        }
    }
    PrintArray(array);
}

Console.Write("после: ");
PrintArray(array);
```

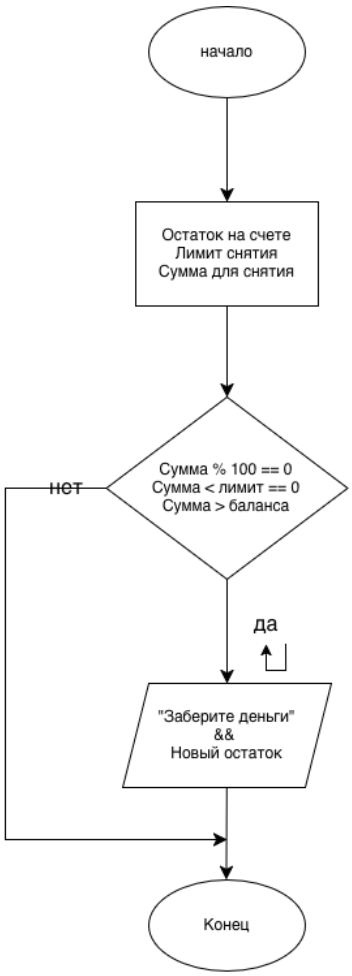
1. Постановка задачи.

«Банкомат»: Пользователь запрашивает сумму для снятия. Алгоритм проверяет, кратна ли сумма 100, не превышает ли лимит и не превышает ли остаток на счете. Если все проверки пройдены, выводит «Заберите деньги» и уменьшает остаток.

2. Контрольный пример.

| **Входные данные** | **Значения** |
|---|---|
| Amount | 1000, 150, 4000 |
| **Выходные данные** | **Значения** |
| Сообщение | Успех, остаток 4000; Ошибка кратности, Недостаточно средств; |

3. Блок схема.



4. Текст программы.

```
Console.WriteLine($"Остаток на счёте: {balance} руб.");
Console.WriteLine($"Лимит снятия:     {limit} руб.\n");

Console.Write("Введите сумму для снятия: ");
if (!double.TryParse(s: Console.ReadLine(), out double amount))
{
    Console.WriteLine("Некорректная сумма.");
    Program.Pause();
    return;
}

if (amount % 100 != 0)
{
    Console.WriteLine("Ошибка: сумма должна быть кратна 100.");
}
else if (amount > limit)
{
    Console.WriteLine($"Ошибка: сумма превышает лимит ({limit} руб.).");
}
else if (amount > balance)
{
    Console.WriteLine("Ошибка: недостаточно средств.");
}
else
{
    balance -= amount;
    Console.WriteLine("Заберите деньги!");
    Console.WriteLine($"Новый остаток: {balance} руб.");
}
```

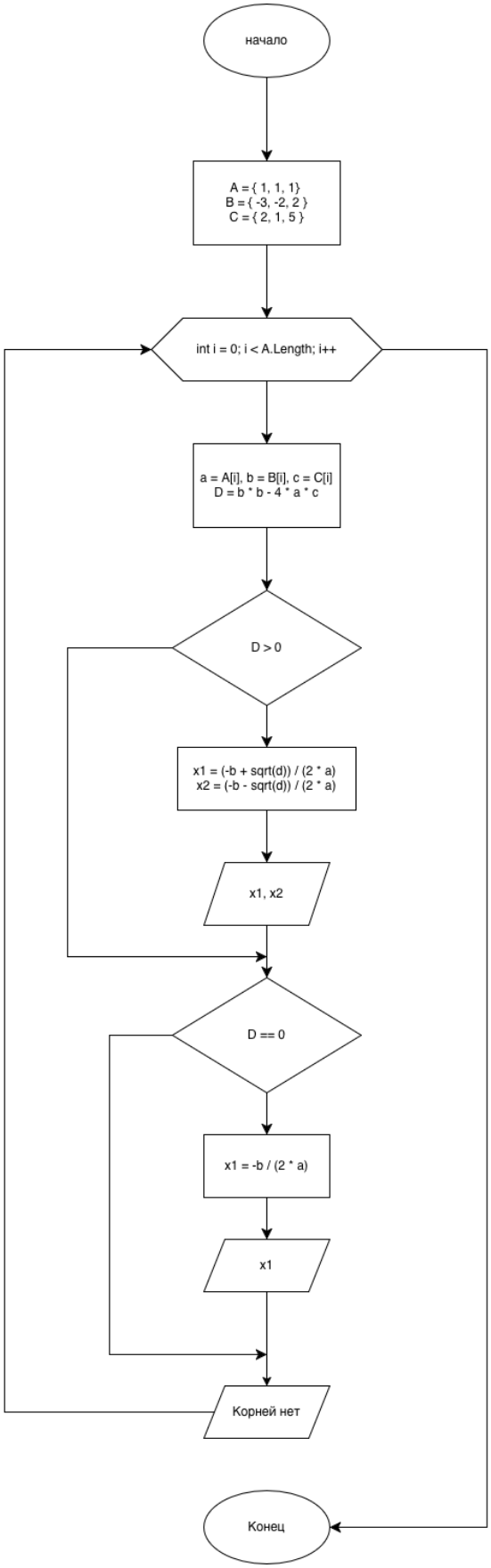
1. Постановка задачи.

«Решение квадратного уравнения»: Алгоритм получает коэффициенты a, b, c. (Массивы коэффициентов). Вычисляет дискриминант (D) и в зависимости от его значения (D>0, D=0, D<0) вычисляет и выводит соответствующие корни или сообщение об их отсутствии.

2. Контрольный пример.

| **Входные данные** | **Значения** |
|---|---|
| A, b, c | 1, -3, 2 |
| A, b, c | 1, -2, 1 |
| A, b, c | 1, 2, 5 |
| **Выходные данные** | **Значения** |
| X1, X2 | 2, 1 |
| X1 | 1 |
| Корней нет | Корней нет |

3. Блок схема.


начало
A = { 1, 1, 1}
B = { -3, -2, 2 }
C = { 2, 1, 5 }
int i = 0; i < A.Length; i++
a = A[i], b = B[i], c = C[i]
D = b * b - 4 * a * c
D > 0
x1 = (-b + sqrt(d)) / (2 * a)
x2 = (-b - sqrt(d)) / (2 * a)
x1, x2
D == 0
x1 = -b / (2 * a)
x1
Корней нет
Конец

4. Текст программы.

```
double[] A = { 1, 1, 1 };
double[] B = { -3, -2, 2 };
double[] C = { 2, 1, 5 };

for (int i = 0; i < A.Length; i++)
{
    double a = A[i], b = B[i], c = C[i];
    Console.WriteLine($"--- Уравнение: {a}x² + {b}x + {c} = 0 ---");

    double D = b * b - 4 * a * c;
    Console.WriteLine($"D = {D}");

    if (D > 0)
    {
        double x1 = (-b + Math.Sqrt(D)) / (2 * a);
        double x2 = (-b - Math.Sqrt(D)) / (2 * a);
        Console.WriteLine($"Два корня: x1 = {x1}, x2 = {x2}");
    }
    else if (D == 0)
    {
        double x1 = -b / (2 * a);
        Console.WriteLine($"Один корень: x = {x1}");
    }
    else
    {
        Console.WriteLine("Корней нет (D < 0)");
    }
    Console.WriteLine();
}
```

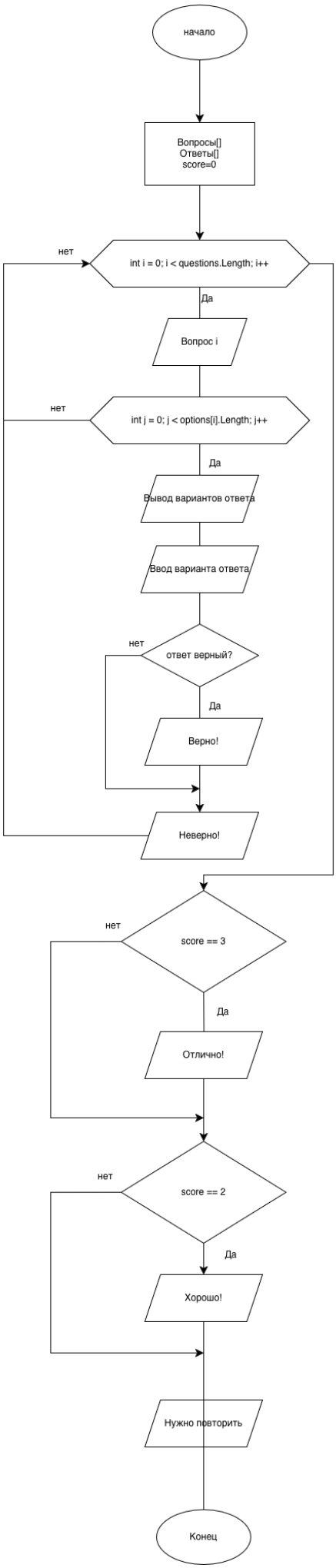
1. Постановка задачи.

«Викторина»: Алгоритм задает пользователю 3 вопроса с 3 вариантами ответов один правильный. Считает количество правильных ответов и в конце выдает оценку (например, «Отлично!», «Нужно повторить материал»).

2. Контрольный пример.

| **Входные данные** | **Значения** |
|---|---|
| Варианты ответов | 1, 1, 1; 1, 2, 3; 2, 2, 2; |
| **Выходные данные** | **Значения** |
| Сообщение | Отлично, Хорошо, Нужно повторить |

3. Блок схема.


начало
Вопросы[]
Ответы[]
score=0
нет
int i = 0; i < questions.Length; i++
Да
Вопрос i
нет
int j = 0; j < options[i].Length; j++
Да
Вывод вариантов ответа
Ввод варианта ответа
нет
ответ верный?
Да
Верно!
Неверно!
нет
score == 3
Да
Отлично!
нет
score == 2
Да
Хорошо!
Нужно повторить
Конец

4. Текст программы.

```
for (int i = 0; i < questions.Length; i++)
{
    Console.WriteLine($"Вопрос {i + 1}: {questions[i]}");
    for (int j = 0; j < options[i].Length; j++)
        Console.WriteLine($"  {j + 1}) {options[i][j]}");

    Console.Write("Ваш ответ (1/2/3): ");
    if (!int.TryParse(s: Console.ReadLine(), out int ans) || ans < 1 || ans > 3)
    {
        Console.WriteLine("Некорректный ввод. Пропуск вопроса.");
        continue;
    }
    int answer = ans - 1;

    if (answer == correctAnswers[i])
    {
        Console.WriteLine("Верно!\n");
        score++;
    }
    else
    {
        Console.WriteLine($"Неверно! Правильный ответ: {options[i][correctAnswers[i]]}\n");
    }
}
```

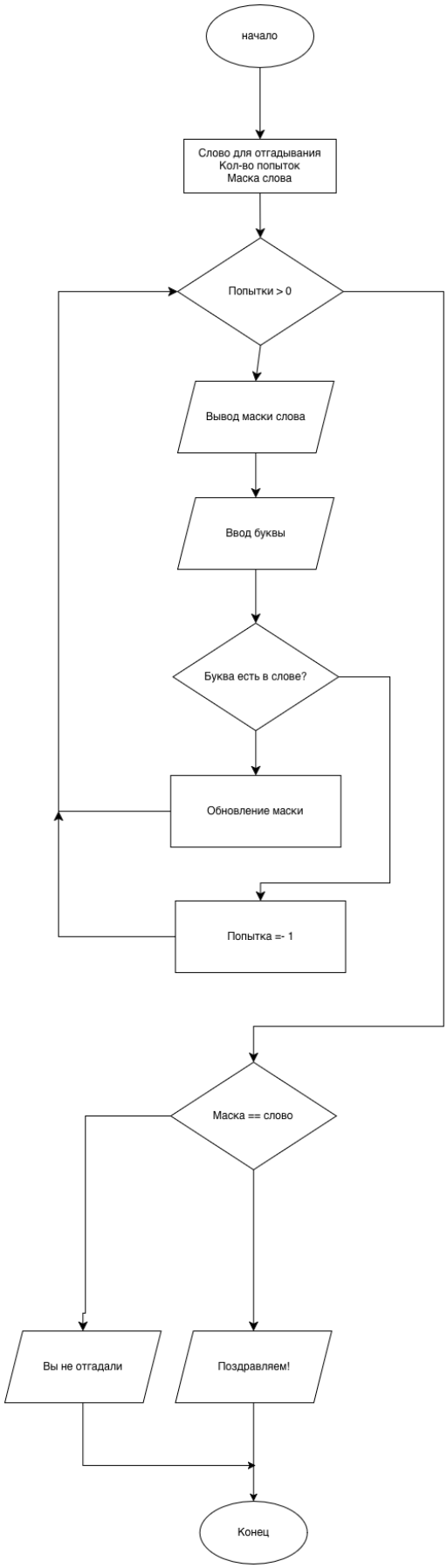
1. Постановка задачи.

«Игра «Угадай слово»»: Загадано слово (например, «КОМПЬЮТЕР»). Пользователь по буквам пытается его угадать. Алгоритм показывает текущее состояние слова (К _ _ _ _ Ю _ Е _) и количество оставшихся попыток. Количество попыток- это кол-во нераскрытых букв.

2. Контрольный пример.

| **Входные данные** | **Значения** |
|---|---|
| Введенная буква | К, А |
| **Выходные данные** | **Значения** |
| Сообщение | Верно, _ => К; Нет такой, попытка -1 |

3. Блок схема.


начало
Слово для отгадывания
Кол-во попыток
Маска слова
Попытки > 0
Вывод маски слова
Ввод буквы
Буква есть в слове?
Обновление маски
Попытка =- 1
Маска == слово
Вы не отгадали
Поздравляем!
Конец

4. Текст программы.

```
string word = "КОМПЬЮТЕР";
string guessed = new string('_', word.Length);
int attempts = word.Length;

while (attempts > 0 && guessed.Contains('_'))
{
    Console.WriteLine($"{guessed} | Попыток: {attempts}");
    Console.Write("Буква: ");

    char letter = Console.ReadLine()?.ToUpper()[0] ?? '\0';
    if (letter == '\0') continue;

    bool found = false;
    for (int i = 0; i < word.Length; i++)
        if (word[i] == letter && guessed[i] == '_')
        {
            guessed = guessed.Remove(startIndex: i, count: 1).Insert(i, letter.ToString());
            found = true;
        }

    attempts -= found ? 0 : 1;
    Console.WriteLine(found ? "Верно!" : "Нет такой буквы!\n");
}

Console.WriteLine($"\n{guessed}");
Console.WriteLine(guessed == word ? "Поздравляем!" : $"Увы. Слово: {word}");
Program.Pause();
```

1. Постановка задачи.

«Лифт»: Смоделируйте работу лифта в 9-этажном доме. Алгоритм запрашивает текущий этаж и целевой этаж. Учитывает направление движения, «открывает/закрывает двери» (сообщения) и отображает текущий этаж в процессе «движения» (цикл с задержкой).

2. Контрольный пример.

| **Входные данные** | **Значения** |
|---|---|
| Введенный этаж | 5, 1, 0 |
| **Выходные данные** | **Значения** |
| Сообщение | 1-5 вверх, вы уже на нужном этаже, неверный этаж |

3. Блок схема.

Начало
Текущий этаж = 1
Целевой этаж
Текущий этаж == целевой этаж?
Нет
Определение направления
Да
Вы на месте
Двери открываются
Двери закрываются
Текущий этаж == целевой этаж?
Нет
Да
Прибытие. Двери открываются
Переход на этаж
Начало

4. Текст программы.

```
int currentFloor = 1;
Console.WriteLine($"Лифт сейчас на этаже: {currentFloor}");
Console.Write("Введите целевой этаж (1-9): ");
if (!int.TryParse(s: Console.ReadLine(), out int targetFloor) || targetFloor < 1 || targetFloor > 9)
{
    Console.WriteLine("Неверный этаж!");
    Program.Pause();
    return;
}

if (targetFloor == currentFloor)
{
    Console.WriteLine("Вы уже на нужном этаже!");
    Program.Pause();
    return;
}

int direction = targetFloor > currentFloor ? 1 : -1;
string dirName = direction == 1 ? "вверх ↑" : "вниз ↓";
```

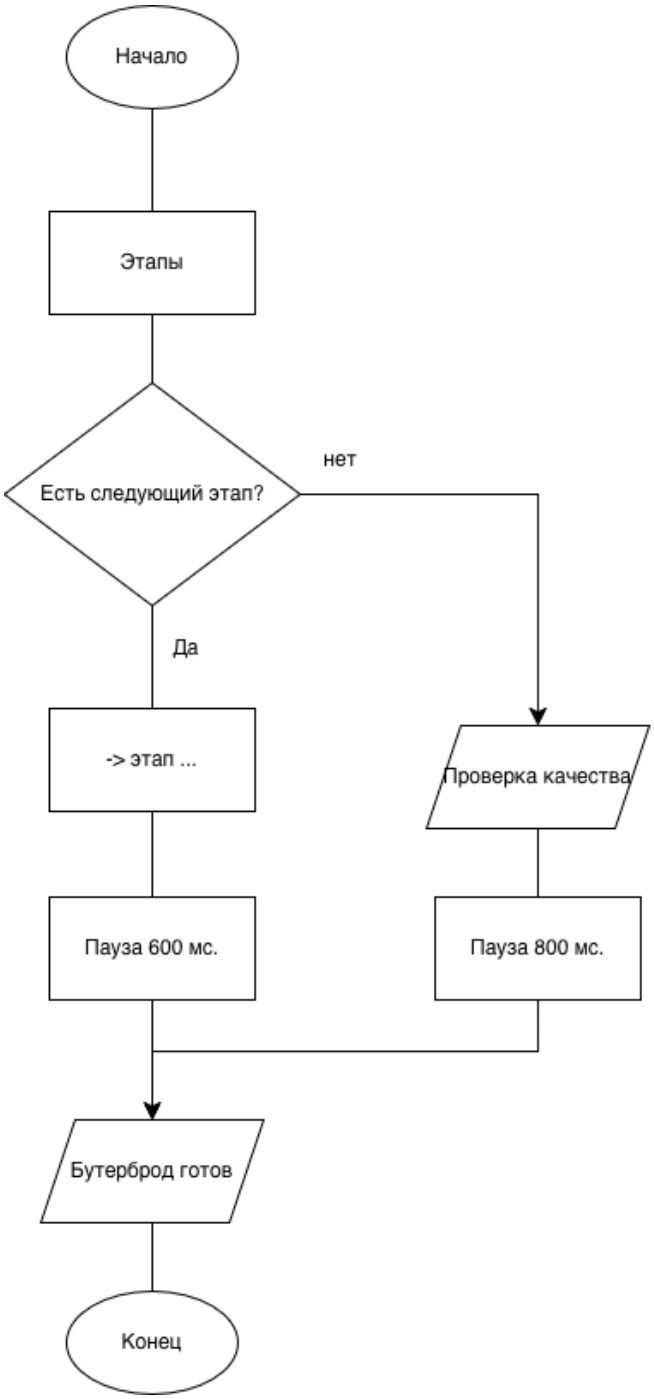
1. Постановка задачи.

«Конвейер сборки»: Изобразите процесс сборки устройства (например, бутерброда или простого робота) из 4-5 компонентов. На конвейере последовательно выполняются действия, но только если предыдущий этап завершен успешно. Добавьте этап проверки качества.

2. Контрольный пример.

| **Входные данные** | **Значения** |
|---|---|
| Этапы | Готовые 5 этапов |
| **Выходные данные** | **Значения** |
| Сообщение | Бутерброд готов. |

3. Блок схема.


Начало
Этапы
Есть следующий этап?
нет
Да
-> этап ...
Проверка качества
Пауза 600 мс.
Пауза 800 мс.
Бутерброд готов
Конец

4. Текст программы.

```
string[] stages = {
    "Кладём хлеб",
    "Намазываем масло",
    "Кладём сыр",
    "Добавляем ветчину",
    "Накрываем хлебом"
};


foreach (string stage in stages)
{
    Console.WriteLine($"→ {stage}...");
    Thread.Sleep( millisecondsTimeout: 600);
}
```